DEVOPS & AI ENGINEERING HANDBOOK

Claude CLI (Claude Code) — Manual Maestro

Taxonomía completa de comandos por categorías, optimización de flujos autónomos y conexión local ilimitada.

Versión CLI: **2.1.296+** Compatibilidad: **macOS / Linux / WSL** Enrutador: **OmniRoute & Multi-Model** Edición: **Oficial Octubre 2026**

1 Invocación y Modos de Ejecución

Formas de inicializar el motor de Claude Code en terminal según la naturaleza interactiva o desatendida de la tarea.

COMANDO / FLAG	MODO	FUNCIÓN TÉCNICA	CASO DE USO
`claude`	REPL	Inicia sesión interactiva conversacional en el directorio actual.	Desarrollo diario colaborativo.
`claude "prompt"`	DIRECTO	Abre sesión interactiva ejecutando la instrucción inicial de inmediato.	Lanzar tareas concretas al vuelo.
`claude -p "prompt"` `--print`	ONE-SHOT	Modo no interactivo (headless). Ejecuta la tarea, imprime y sale con código 0/1.	Scripts Bash, CI/CD, Crons y Pipings.
`claude --bare`	MINIMAL	Modo ultrarrápido sin hooks, plugins, LSP, keychain ni auto-memory.	Servidores remotos y ejecuciones rápidas.
`claude --safe-mode`	SEGURO	Inicia con todas las personalizaciones (CLAUDE.md, MCP, plugins) apagadas.	Resolución de conflictos o fallos.
`claude --json`	FORMATO	Emite la respuesta en JSON estructurado para parsing automático.	Filtrado directo con `jq`.

```
# Pipeline headless: Resumen automático de cambios con Conventional Commits
git diff HEAD~1 | claude -p "Resume los cambios y genera el mensaje de commit"
```

2 Gestión de Sesiones, Continuidad y Resumen

Comandos para mantener el hilo del trabajo entre cierres de terminal, cambios de máquina o transferencias al escritorio.

COMANDO / FLAG	FUNCIÓN Y ALCANCE	EJEMPLO DE USO
`claude -c` / `--continue`	Reanuda automáticamente la última conversación activa en la carpeta actual.	`claude -c`
`claude -r [id|query]` `--resume`	Abre el selector interactivo con buscador de sesiones pasadas o carga por ID.	`claude -r "auth-fix"`
`claude --session-id <uuid>`	Fuerza un identificador UUID explícito para trazabilidad y registros.	`claude --session-id $UUID`
`claude --teleport [session]`	Teletransporta y sincroniza la sesión actual con la nube o claude.ai.	`claude --teleport`
`claude --desktop`	Abre la sesión de la terminal directamente en la interfaz de Claude Desktop.	`claude -c --desktop`
`claude purge [path]`	Elimina caché, transcripciones, tareas guardadas y estados del proyecto.	`claude purge .`

3 Agentes en Segundo Plano (Background & Daemons)

Permite delegar tareas pesadas o de análisis masivo a subagentes autónomos sin bloquear la terminal activa.

COMANDO	DESCRIPCIÓN OPERATIVA	RESULTADO
`claude --bg "tarea"`	Lanza la tarea en segundo plano e imprime el ID único de la sesión inmediatamente.	Proceso en background.
`claude agents`	Lista todos los agentes en ejecución, memoria de trabajo y estado de avance.	Monitor de tareas.
`claude attach <id>`	Conecta la terminal interactiva a la sesión del agente para tomar el control.	Retoma en vivo.
`claude logs <id>`	Imprime la salida reciente del terminal generada por el agente daemon.	Lectura de trazas.
`claude stop <id>` / `kill`	Detiene el agente conservando el contexto para reanudarlo después.	Pausa segura.
`claude rm <id>`	Elimina la sesión en segundo plano y limpia su árbol de trabajo temporal.	Liberación de recursos.
`claude respawn <id>`	Reinicia el agente en segundo plano actualizándolo con la versión más reciente.	Reinicio de sesión.

⚡ **Flujo Recomendado de Trabajo Asíncrono**

Dispara: `claude --bg "Ejecuta los tests de integración y corrige los timeouts en los endpoints"`
Monitorea el progreso con `claude logs <id>` y engánchate cuando termine con `claude attach <id>`.

4 Aislamiento de Código: Git Worktrees & Tmux

Permite a Claude crear ramas paralelas de forma segura sin tocar tu código local en desarrollo.

COMANDO / OPCIÓN	FUNCIONALIDAD	VENTAJA
`claude -w [nombre]` `--worktree`	Crea un nuevo git worktree aislado y corre la sesión completamente adentro.	Cero riesgo de sobreescritura local.
`claude -w fix-api --tmux`	Combina el git worktree con una sesión nativa de tmux o panel iTerm2.	Vista paralela en pantalla dividida.
`claude --add-dir <dirs...>`	Otorga acceso de lectura/escritura a directorios adicionales fuera del root.	Ideal para monorrepositorios.

5 Seguridad, Permisos y Sandboxing

Políticas de ejecución y delimitación de herramientas para operaciones desatendidas o de alta seguridad.

BANDERA / OPCIÓN	DESCRIPCIÓN	NIVEL DE RIESGO
`--dangerously-skip-permissions`	Omite toda solicitud de confirmación para ejecutar comandos o editar archivos.	ALTO (SANDBOX)
`--allowedTools "Bash(git *) Edit"`	Habilita estrictamente las herramientas indicadas mediante lista blanca.	CONTROLADO
`--tools ""` / `--tools default`	Desactiva todas las herramientas (modo solo texto) o carga el set nativo.	SOLO LECTURA
`--quarantine`	Modo de máxima protección: bloquea escrituras en `.git` y configs críticas.	BLINDADO

6 Administración del Ecosistema MCP (Model Context Protocol)

Conecta Claude CLI con bases de datos, APIs de terceros, navegadores y servidores locales mediante el estándar abierto MCP.

COMANDO	FUNCIÓN PRINCIPAL	TRANSPORTE
`claude mcp list`	Lista todos los servidores MCP configurados, aprobados y su estado de conexión.	Diagnóstico
`claude mcp add <name> -- <cmd>`	Registra un servidor MCP basado en proceso local estándar (stdio).	STDIO
`claude mcp add --transport http <name> <url>`	Conecta a un servidor MCP remoto mediante protocolo HTTP / SSE.	HTTP / SSE
`claude mcp add-from-claude-desktop`	Importa automáticamente los servidores MCP configurados en la app Desktop.	Importación
`claude mcp get <name>`	Muestra detalles, herramientas expuestas, variables y health-check.	Inspección
`claude mcp remove <name>`	Elimina el registro de un servidor MCP del entorno del proyecto o global.	Desinstalación
`claude mcp serve`	Inicia el propio Claude Code como servidor MCP para ser consumido por otras IAs.	Servidor Bridge
`claude --strict-mcp-config`	Usa exclusivamente los MCPs indicados en el flag, ignorando la configuración global.	Aislamiento

```
# Registrar base de datos SQLite y servidor de GitHub mediante MCP
claude mcp add sqlite -- npx -y @modelcontextprotocol/server-sqlite /path/to/database.db
claude mcp add github -e GITHUB_PERSONAL_ACCESS_TOKEN=$TOKEN -- npx -y @modelcontextprotocol/server-github
```

7 Gestión de Plugins y Extensiones

Administra paquetes de habilidades (skills), hooks de ciclo de vida y herramientas empaquetadas.

COMANDO	DESCRIPCIÓN	ACCIÓN
`claude plugin list`	Muestra todos los plugins instalados y su estado activo/inactivo.	Verificación
`claude plugin install <plugin>`	Instala un nuevo plugin desde el registro oficial o repositorio.	Instalación
`claude plugin enable / disable <p>`	Habilita o suspende temporalmente una extensión sin desinstalarla.	Alternancia
`claude plugin remove <plugin>`	Desinstala por completo el paquete de extensión del sistema.	Limpieza
`claude --plugin-dir <path>`	Carga un plugin local en desarrollo para depuración inmediata.	Dev local

8 Comandos Slash Interactivos en Sesión (Prompt REPL)

Atajos que se teclean directamente dentro del chat de Claude CLI para controlar la sesión, reducir costos y depurar en caliente.

COMANDO SLASH	EFECTO EN LA SESIÓN	IMPACTO EN CONTEXTO / COSTO
`/help`	Muestra el menú rápido de comandos interactivos y sintaxis.	0 tokens (Local)
`/init`	Analiza la arquitectura del repositorio y genera el archivo directriz `CLAUDE.md`.	Guarda reglas persistentes
`/compact`	Comprime y resume el historial de conversación para liberar ventana de contexto.	📉 Reduce 70-85% tokens
`/clear`	Borra el historial completo de la sesión y reinicia la memoria a cero.	🧹 Restablece a 0 tokens
`/cost`	Desglosa el consumo exacto de tokens (Input vs Output) y el costo acumulado.	0 tokens (Telemetría)
`/doctor`	Ejecuta un diagnóstico profundo de dependencias, git, Node, llaves y puertos.	0 tokens (Chequeo interno)
`/config`	Abre el panel interactivo de preferencias, temas, modelos y atajos de teclado.	Configuración visual
`/memory`	Muestra y permite editar las directivas y notas persistentes que Claude ha retenido.	Edición de memoria
`/model`	Permite cambiar de modelo de IA en caliente sin cerrar la conversación actual.	Cambio dinámico
`/pr`	Revisa los cambios de la rama actual y genera el título y descripción del PR.	Automatización git
`/review`	Dispara una revisión estricta de código del diff antes de hacer commit.	Control de calidad
`/bug`	Captura telemetría de depuración para diagnosticar errores de la sesión.	Reporte técnico

🧠 **Arquitectura del Archivo CLAUDE.md (Memoria del Proyecto)**

Ejecuta `/init` una sola vez en cada nuevo repositorio. Claude creará un archivo `CLAUDE.md` en la raíz que almacena: comandos habituales de compilación (build/test/lint), estilo de código del equipo y librerías prohibidas o recomendadas. Claude lee este archivo en cada arranque sin necesidad de que se lo recuerdes en los prompts.

9 Integración con OmniRoute: Claude CLI 100% Gratis & Ilimitado

Aprovecha la capacidad estándar de Claude CLI para apuntar a un gateway local y desbloquear +300 modelos de IA sin suscripción fija.

```
# 1. Configuración de entorno en terminal para apuntar al router local:
export ANTHROPIC_BASE_URL="http://localhost:20128"
export ANTHROPIC_API_KEY="sk-omniroute-local-token"

# 2. Inicia Claude CLI consumiendo tus modelos locales (Ollama) o cloud gratuitos (Groq, Gemini):
claude

# 3. Alias definitivo en ~/.zshrc para abrir Claude Gratis con un comando:
alias claude-free='ANTHROPIC_BASE_URL="http://localhost:20128" claude'
```

10 Recetario Power-User de Automatización y Piping

```
# Generar suite de pruebas automática
claude -p "Crea pruebas completas en pytest para src/auth.py guardando en tests/test_auth.py"

# Depurar logs de producción directamente desde la terminal
tail -n 80 /var/log/app.log | claude -p "Explica el error y genera la corrección exacta"

# Revisión profunda multi-agente de toda la rama
claude ultrareview main
```

★ Matriz de Decisión Rápida (Cheat Sheet)

OBJETIVO DESEADO	COMANDO PRINCIPAL	ATAJO / PARÁMETRO CLAVE
Programar interactivamente en un repo	`claude`	Usa `/init` la primera vez.
Reanudar la última sesión sin perder contexto	`claude -c`	`--continue`
Ejecutar un script desatendido (CI/CD)	`claude -p "prompt"`	`--dangerously-skip-permissions`
Lanzar tarea pesada sin bloquear la consola	`claude --bg "tarea"`	Ver con `claude agents`.
Hacer cambios de prueba sin tocar la rama	`claude -w nombre-rama`	Git worktree aislado.
Reducir costos cuando se satura la memoria	`/compact`	Dentro del chat activo.
Diagnosticar fallos del sistema y dependencias	`claude doctor`	O `/doctor` dentro del REPL.